

Ensemble Methods

Hrachya Asatryan

Decision Trees Recap

Main intuition

Create a tree based structure that splits based on every feature.

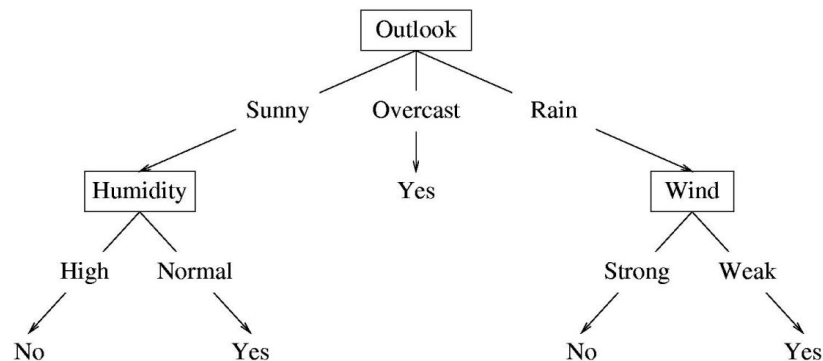
What is our goal?

Our goal is to fit the data the best we can, while we make no assumption about the underlying function.

How do we do that?

We split the data into branches in a way, that maximizes our information gain.

Predictors				Response
Outlook	Temperature	Humidity	Wind	Class
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No



Problem Formulation

Main assumptions

1. Data can be split into subsets based on the input variables from the data features.
2. The split can stand alone and is able to maximize information gain.
3. These splits are binary and this means each split is formed into two subsets from a single input feature.

Advantages

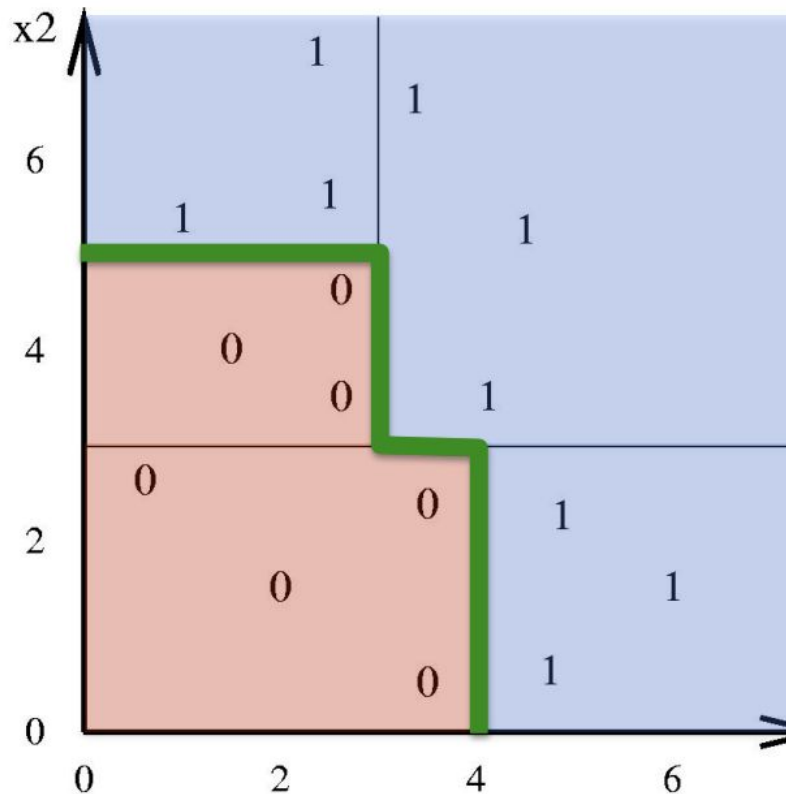
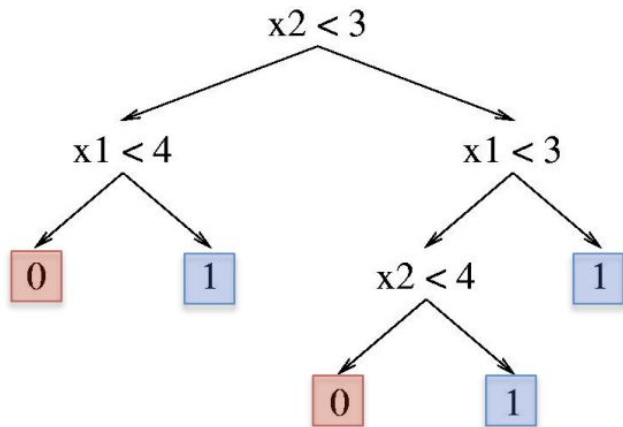
- Can perform functions on both categorical and numerical data, making it versatile.
- Adding new features is straightforward.
- it requires minimal data preprocessing since it's robust against outliers.
- In conjunction with ensemble methods, it can be used to build more larger classifiers.
- It doesn't require scaling or normalization of the data.

Disadvantages

- A small change in the data can cause a large change in the decision tree structure causing instability.
- The calculation can be far more complex compared to other algorithms, taking more time to train the models.
- Trees are prone to overfitting.
- Require some kind of measurement as to how well they are doing.
- Can be biased, if some classes dominate.

Decision Boundary

Decision trees divide the feature space into axis-parallel rectangles. Each rectangular region is labeled with one label or a probability distribution over labels. Trees can encapsulate any boolean function, worst case they will need exponentially many nodes.



Algorithm

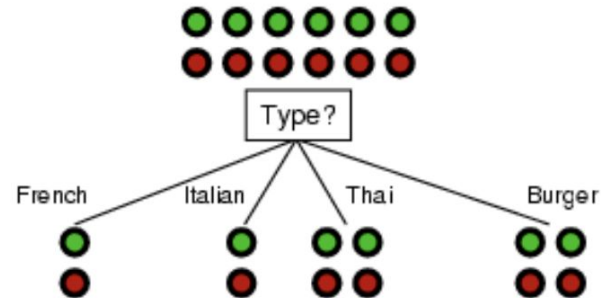
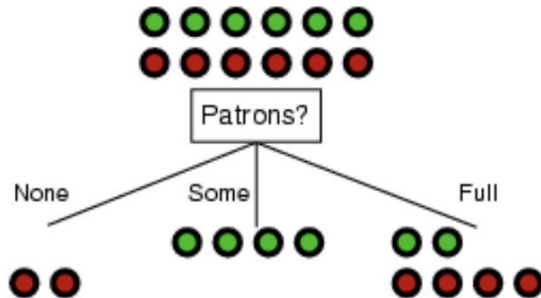
How do I construct a tree?

Given that **A** is our best decision attribute for the next node:

1. Assign **A** as a decision attribute.
2. For each value of **A** create a new descendant of node.
3. Sort training examples to the leaf nodes.
4. If the training examples are perfectly classified, stop. Else recurse over new leaf nodes.

How do we choose the best decision attribute?

A good attribute splits the examples into (ideally) uniform subsets:

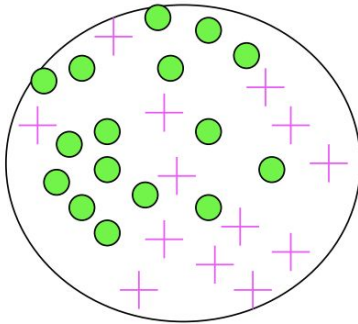


Algorithm

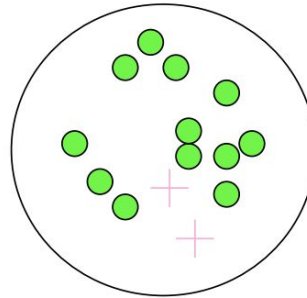
What is impurity?

As the name suggests, we measure how pure our data is, given the split, which in turn shows how much information we have gained after the split.

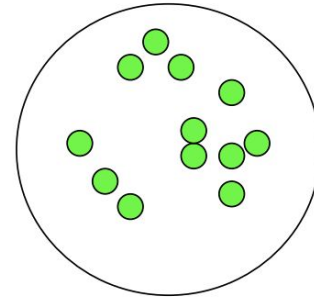
Very impure group



Less impure



**Minimum
impurity**



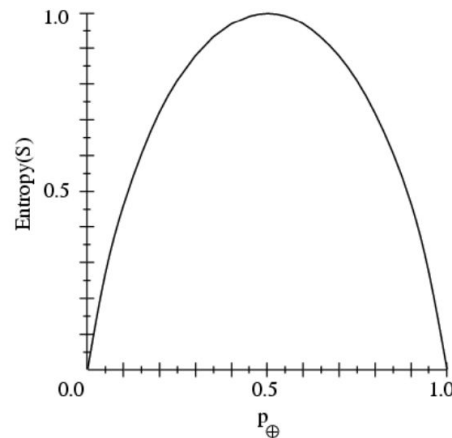
Algorithm

What is impurity?

We measure impurity using different methods, one of them is Entropy:

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

Thus given the formulation, we get a maximum entropy,
When our split contains about the same number of
elements from each class.



Algorithm

From Entropy to Information Gain

We want to determine which attribute in a given set of training feature vectors is the most useful for discriminating between the classes to be learned. Information gain tells us how important a given attribute of the feature vectors is. We will use it to decide the ordering of attributes the nodes of a decision tree.

Thus for **Gain** we will have:

$$I(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

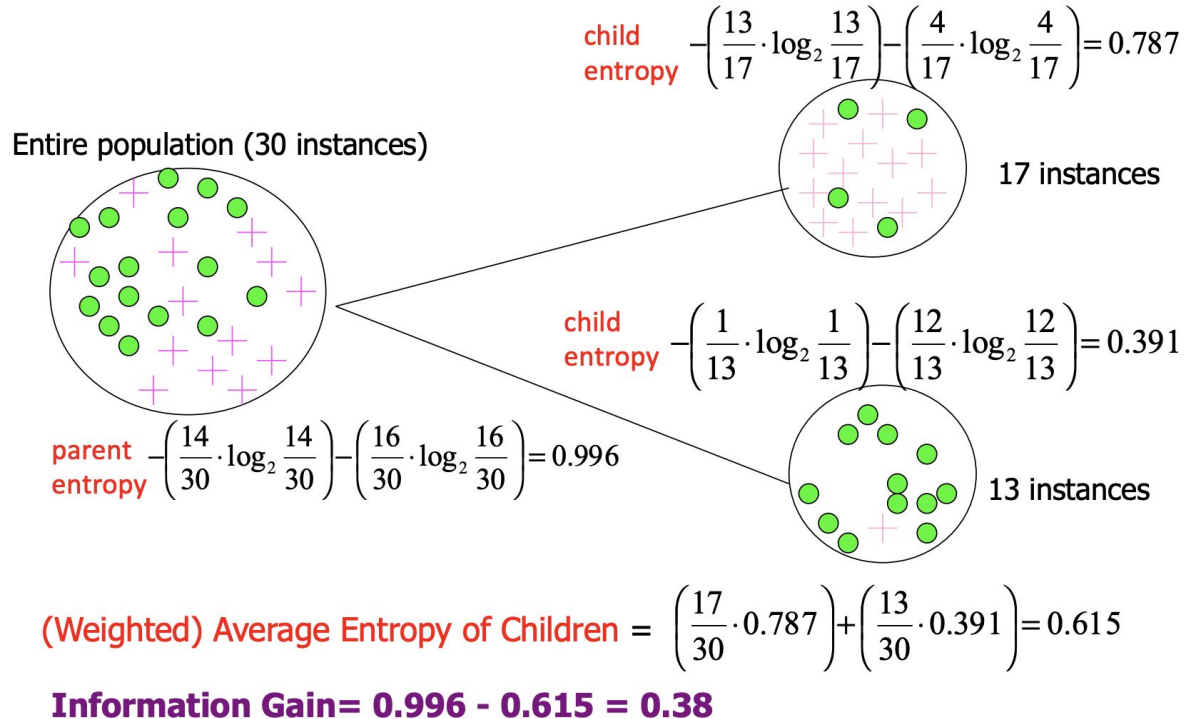
$$H(X|Y = v) = - \sum_{i=1}^n P(X = i|Y = v) \log_2 P(X = i|Y = v)$$

$$H(X|Y) = \sum_{v \in \text{values}(Y)} P(Y = v) H(X|Y = v)$$

Algorithm

Calculating Information Gain

Information Gain = entropy(parent) – [average entropy(children)]



Algorithm

Is our tree any good?

Our ideal tree should be:

- Not too small: need to handle important but possibly subtle distinctions in data
- Not too big:
 - ◆ Avoid overfitting training examples.
 - We need enough samples in each region to confidently determine the output.
 - ◆ Computational efficiency (avoid redundant, spurious attributes)
 - ◆ Human interpretability
- Occam's Razor: find the simplest hypothesis that fits the observations
- We desire small trees with informative nodes near the root

We might encounter problems such as:

- Having exponentially less data at lower levels
- A large tree can overfit the data
- Greedy algorithms do not necessarily yield the global optimum
- Mistakes at top-level propagate down

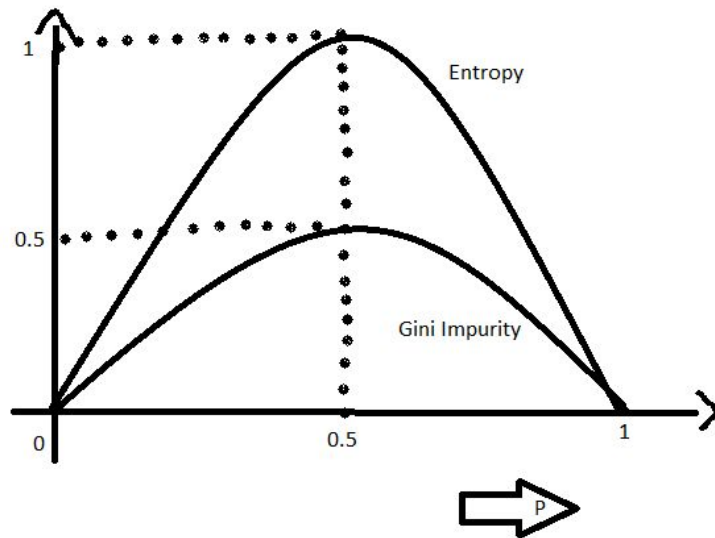
Gini Impurity

Are there other ways to split?

As Entropy can be computationally cost heavy, we can devise other functions that behave similarly and take less resources. One of them is the Gini impurity.

The Gini Index is the probability of misclassifying a randomly chosen element in a set, while entropy measures the amount of uncertainty or randomness in a set. It is a linear measure and can be interpreted as the expected error rate in a classifier.

However it is less robust and more sensitive than entropy.



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

History of Decision Trees

ID3 (1986):

- Uses **Information Gain / Entropy**.
- Only handles categorical data.
- Creates multi-way splits (one branch per category).

C4.5 (1993):

- The successor to ID3.
- Handles continuous/numerical data via thresholding.
- Uses **Gain Ratio** to penalize features with too many unique values.

CART (1984 - Modern Standard):

- Classification and Regression Trees.
- Uses **Gini Impurity** (Classification) or **MSE** (Regression).
- **Strictly Binary**: Every split only ever has two branches (Yes/No).
- *Note: This is the default algorithm in Scikit-Learn and the basis for our Ensembles!*

Cost-Complexity Pruning

- The strategy is to grow a large tree T_0 by stopping the splitting process only when some minimum node size is reached.
- Then this large tree is pruned using cost-complexity pruning,
- We index terminal nodes by m , with node m representing region R_m .
- Let $|T|$ denote the number of terminal nodes in T .
- We define the cost complexity criterion

$$C_\alpha(T) = \sum_{m=1}^{|T|} N_m Q_m(T) + \alpha |T|.$$

where Q_m is the node impurity

- The idea is to find, for each α , the subtree $T_\alpha \subseteq T_0$ to minimize $C_\alpha(T)$.
- The tuning parameter $\alpha \geq 0$ governs the tradeoff between tree size and its goodness of fit to the data.
- Large values of α result in smaller trees T_α , and conversely for smaller values of α .

Pruning and the Estimation of α

- For each α one can show that there is a unique smallest subtree T_α that minimizes $C_\alpha(T)$.
- To find T_α we use **weakest link pruning**: we successively collapse the internal node that produces the smallest per-node increase in $\sum_m N_m Q_m(T)$, and continue until we produce the single-node tree.
- This gives a (finite) sequence of subtrees, and one can show this sequence must contain T_α .
- Estimation of α is achieved by five- or tenfold cross-validation: we choose the value α^* to minimize the cross-validated sum of squares. Our final tree is T_{α^*} .

Ensemble Methods

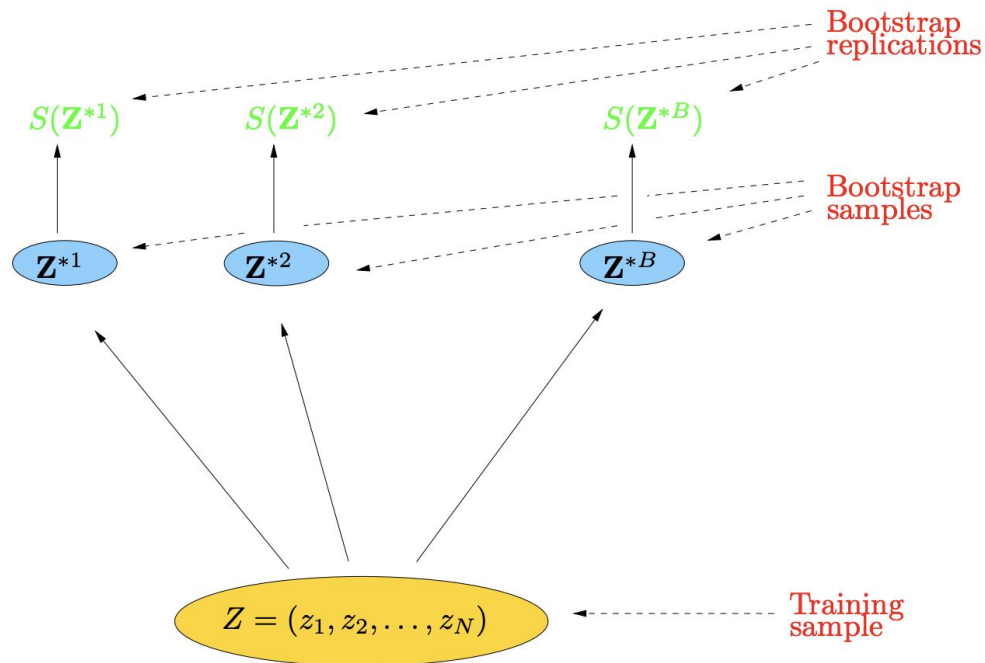
Bootstrap

Bootstrap is a general tool for assessing statistical accuracy. Suppose we have a model fit to a set of training data. We denote the training set by $Z = (z_1, z_2, \dots, z_N)$ where $z_i = (x_i, y_i)$.

The basic idea is to randomly draw datasets with replacement from the training data, each sample the same size as the original training set. This is done B times ($B = 100$ say), producing B bootstrap datasets.

Then we refit the model to each of the bootstrap datasets, and examine the behavior of the fits over the B replications

Bootstrap



Bagging

How do I Bag?

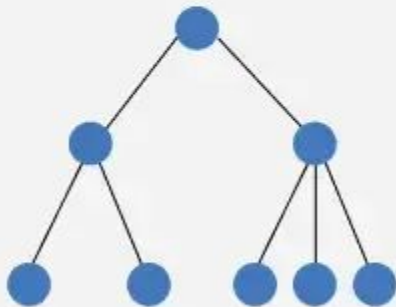
In **Bootstrap Aggregating**, weak learners are trained in parallel and the model combines them for determining the average. It decreases the variance and helps to avoid overfitting. It is usually applied to decision tree methods. Bagging is a special case of the model averaging approach. The algorithm is as follow:

1. **Bootstrapping:** Bagging leverages a bootstrapping sampling technique to create diverse samples. This resampling method generates different subsets of the training data set. It does so by selecting data points at random and with replacement. This means that each time you select a data point from the training data set, you are able to select the same instance multiple times. As a result, a value or instance repeated twice (or more) in a sample.
2. **Parallel training:** These bootstrap samples are then trained independently and in parallel with each other using weak or base learners.
3. **Aggregation:** Finally, depending on the task (that is, regression or classification), an average or a majority of the predictions are taken to compute a more accurate estimate. In the case of regression, an average is taken of all the outputs predicted by the individual classifiers; this is known as soft voting. For classification problems, the class with the highest majority of votes is accepted; this is known as hard voting or majority voting.

Random Forests

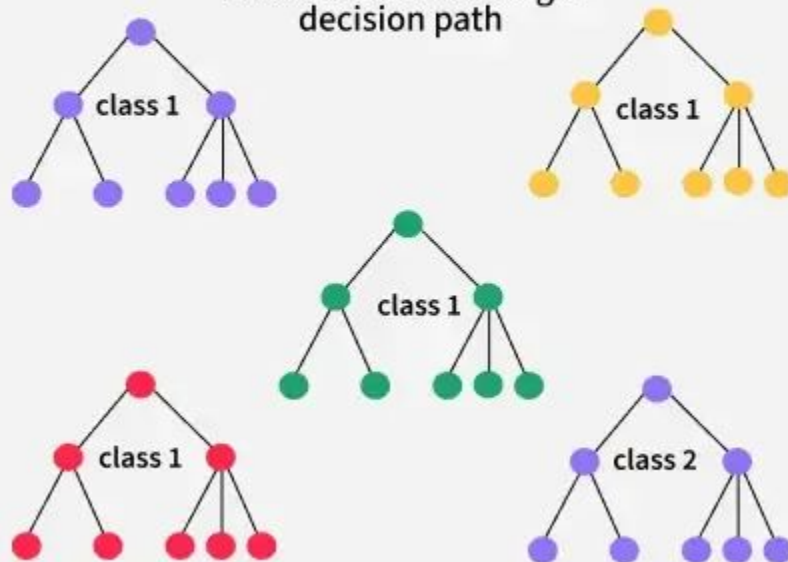
Single Decision Tree

Ensemble of trees for more accurate and robust prediction



Random Forest

Prediction from a single decision path



Random Forests

How does the algorithm work?

1. **Create Many Decision Trees:** The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different. This means some data points may appear multiple times while others aren't used (around $\frac{1}{3}$ of the data - **out of bag examples**).
2. **Pick Random Features:** When making a split, each tree only considers a random subset of features (typically square root of total features). This helps the trees stay different from each other.
3. **Growing Trees:** Each tree grows using only its bootstrap sample and selected features, making splits until it reaches a stopping point (like pure groups or minimum sample size).
4. **Final Prediction:** All trees vote together for the final prediction. For classification, take the majority vote of class predictions; for regression, average the predicted values from all trees.
5. **Out-of-Bag (OOB) Evaluation:** Remember those samples that didn't get used for training each tree – that leftover $\frac{1}{3}$? Those are our OOB samples. Instead of just ignoring them, Random Forest uses them as a convenient validation set for each tree.
6. **Why It Works Well:** Using random data and features for each tree helps avoid overfitting and makes the overall prediction more accurate and trustworthy.

Boosting

What are the main principles of Boosting?

Boosting methods are focused on iteratively combining weak learners to build a strong learner that can predict more accurate outcomes. As a reminder, a weak learner classifies data slightly better than random guessing. Boosting algorithms can differ in how they create and aggregate weak learners during the sequential process.

Three popular types of boosting methods include:

1. **Adaptive boosting or AdaBoost:** This method operates iteratively, identifying misclassified data points and adjusting their weights to minimize the training error. The model continues to optimize in a sequential fashion until it yields the strongest predictor.
2. **Gradient boosting:** This method works by sequentially adding predictors to an ensemble with each one correcting for the errors of its predecessor. However, instead of changing weights of data points like AdaBoost, the gradient boosting trains on the residual errors of the previous predictor. The name, gradient boosting, is used since it combines the gradient descent algorithm and boosting method.
3. **Extreme gradient boosting or XGBoost:** XGBoost is an implementation of gradient boosting that's designed for computational speed and scale. XGBoost leverages multiple cores on the CPU, allowing for learning to occur in parallel during training.

AdaBoost

Algorithm

AdaBoost is an algorithm for constructing a "strong" classifier as linear combination $f(x) = \sum_{t=1}^T \alpha_t h_t(x)$ of "weak" classifiers $h_t(x)$.

Given: $(x_1, y_1), \dots, (x_m, y_m); x_i \in X, y_i \in \{-1, 1\}$

Initialize weights $D_1(i) = 1/m$

For $t = 1, \dots, T$

1. Call WeakLearn, which returns the weak classifier $h_t : X \rightarrow \{-1, 1\}$ with minimum error w.r.t. distribution D_t
2. Choose $\alpha_t \in R$
3. Given that Z_t is a normalization factor chosen so that D_{t+1} is a distribution update:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Giving us: $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$

AdaBoost

Weak Learn

Call WeakLearn, given distribution D_t ; returns weak classifier $h_t : X \rightarrow \{-1, 1\}$ from $H = \{h(x)\}$

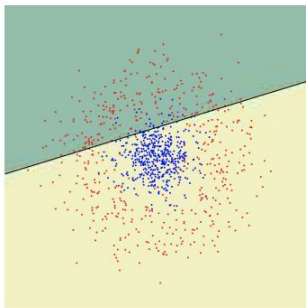
Select a weak classifier with the smallest weighted error:

$$h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$$

If $\epsilon_t \geq 1/2$ then stop.

Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$

Training set



Weak classifier = perceptron

$$\bullet \sim N(0, 1) \quad \bullet \sim \frac{1}{r\sqrt{8\pi^3}} e^{-1/2(r-4)^2}$$

Feature Importance

The Ensemble Problem

A single decision tree is perfectly interpretable. An ensemble of 500 trees is a "black box." How do we know which features actually matter?

Method 1: Mean Decrease Impurity (Gini Importance)

- The algorithm tracks every single time a feature is used to split a node across all 500 trees.
- It calculates how much that specific split reduced the Gini impurity, weighted by how many samples passed through that node.
- *Pros:* Fast, calculated automatically during training.

Method 2: Permutation Importance

- Take a trained model and a test dataset. Record the baseline accuracy.
- Randomly shuffle the values of a single column (e.g., "Age").
- Run the data through the model again. If the accuracy completely crashes, "Age" was critically important. If the accuracy doesn't change, the model wasn't relying on "Age".

Feature Importance

The bias in Mean Decrease Impurity (MDI)

- **The Problem with Gini/MDI:** Standard tree-based importance simply adds up the total impurity reduction for a feature across all nodes.
- **The Bias:** MDI is mathematically biased toward high-cardinality features (continuous variables or categorical variables with many states). A feature with 1,000 unique values has 1,000 different ways to split the data, artificially inflating its chance to reduce impurity, even if it is just random noise.
- **Global vs. Local:** MDI only provides a **Global** explanation ("Feature X is generally important"). It cannot tell us anything about a **Local** prediction ("Why was *this specific patient* classified as high-risk?").
- **The Interaction Blindspot:** If Feature A and Feature B only work well when paired together, MDI often splits the credit unfairly based on whichever one happened to be selected higher up in the tree structures.

Feature Importance

The Shapley Value (1953)

- **The Setup:** Imagine a cooperative game where a coalition of players S generates a total payout $v(S)$. How do we fairly distribute the payout among the players based on their individual contributions?
- **The Challenge:** Players interact. Player i might be useless alone, but incredibly valuable when paired with Player j .
- **The Marginal Contribution:** We calculate the value a player adds to a specific coalition: $\Delta_i(S) = v(S \cup \{i\}) - v(S)$.
- **The Shapley Formula:** Lloyd Shapley proved there is only *one* mathematically fair way to distribute the payout, calculated by averaging the marginal contribution of Player i across all possible permutations of coalitions:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} (v(S \cup \{i\}) - v(S))$$

- *The combinatorial weight $|S|!(|N| - |S| - 1)! / |N|!$ is simply the probability of coalition S forming if players enter the room in a completely random order.*

Feature Importance

SHAP (SHapley Additive exPlanations)

The Translation: To use Shapley values in ML, we redefine the "game":

- **The Players:** The feature values of a specific data point x (e.g., Age=45, BP=130).
- **The Payout (v):** The model's actual prediction for that data point, $f(x)$.
- **The Baseline ($v(\emptyset)$):** The expected value of the model's predictions over the whole dataset, $E[f(X)]$.

The Core Additive Property: SHAP values guarantee that the sum of all feature contributions exactly equals the difference between the actual prediction and the baseline prediction.

$$f(x) = E[f(X)] + \sum_{i=1}^M \phi_i(x)$$

What this means: A SHAP value of +2.5 for Age=45 means that this specific patient's age pushed the model's prediction 2.5 units higher than the population average.

Feature Importance

TreeSHAP

- **The Computational Nightmare:** The exact Shapley formula requires evaluating the model for all 2^M possible feature subsets. For 50 features, this is 1.12×10^{15} evaluations per data point. It is mathematically intractable.
- **TreeSHAP (Lundberg et al., 2017):** A specialized algorithm designed strictly for tree ensembles (Random Forests, XGBoost).
- **The Trick:** Instead of masking features and running the model 2^M times, TreeSHAP pushes the expectations down the decision paths of the trees. If a feature is "missing" in a coalition, the algorithm simply splits the data proportion down *both* sides of the branch, tracking the weighted average.
- **Complexity Reduction:** TreeSHAP reduces the time complexity from $O(2^M)$ to $O(T L D^2)$ (where T = number of trees, L = max leaves, D = max depth). It calculates exact SHAP values in polynomial time.

Feature Importance

Visualizing SHAP values: Forceplots and Beeswarm plots

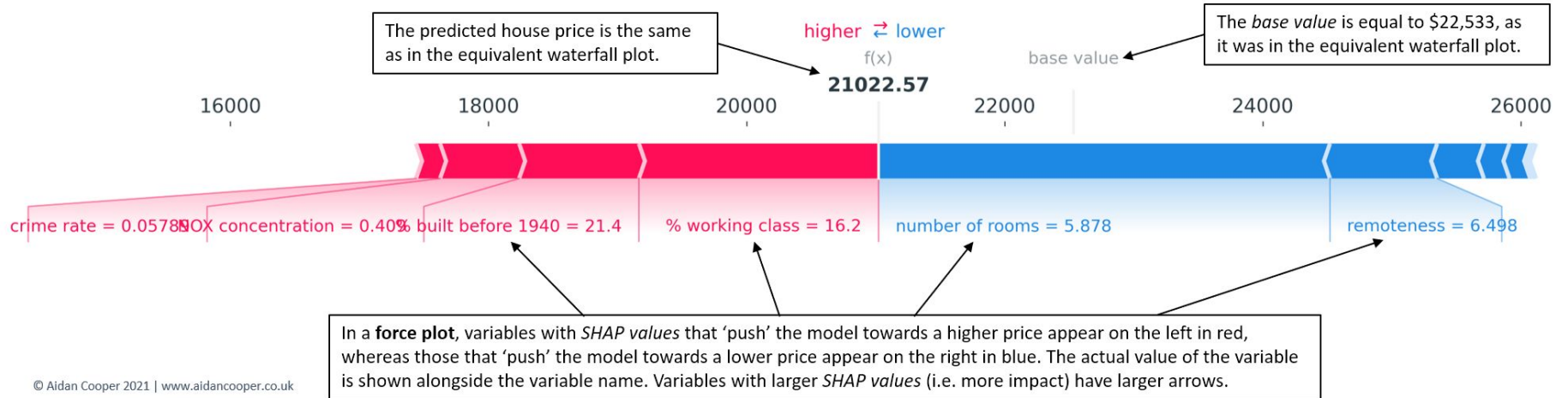
Local Explanation (Force Plot): Used to explain a *single* prediction.

- It shows the base value $E[f(X)]$ and the final output $f(x)$.
- Features with positive SHAP values are red arrows pushing the output right (higher). Features with negative SHAP values are blue arrows pushing it left (lower). The length of the arrow is the exact SHAP value.

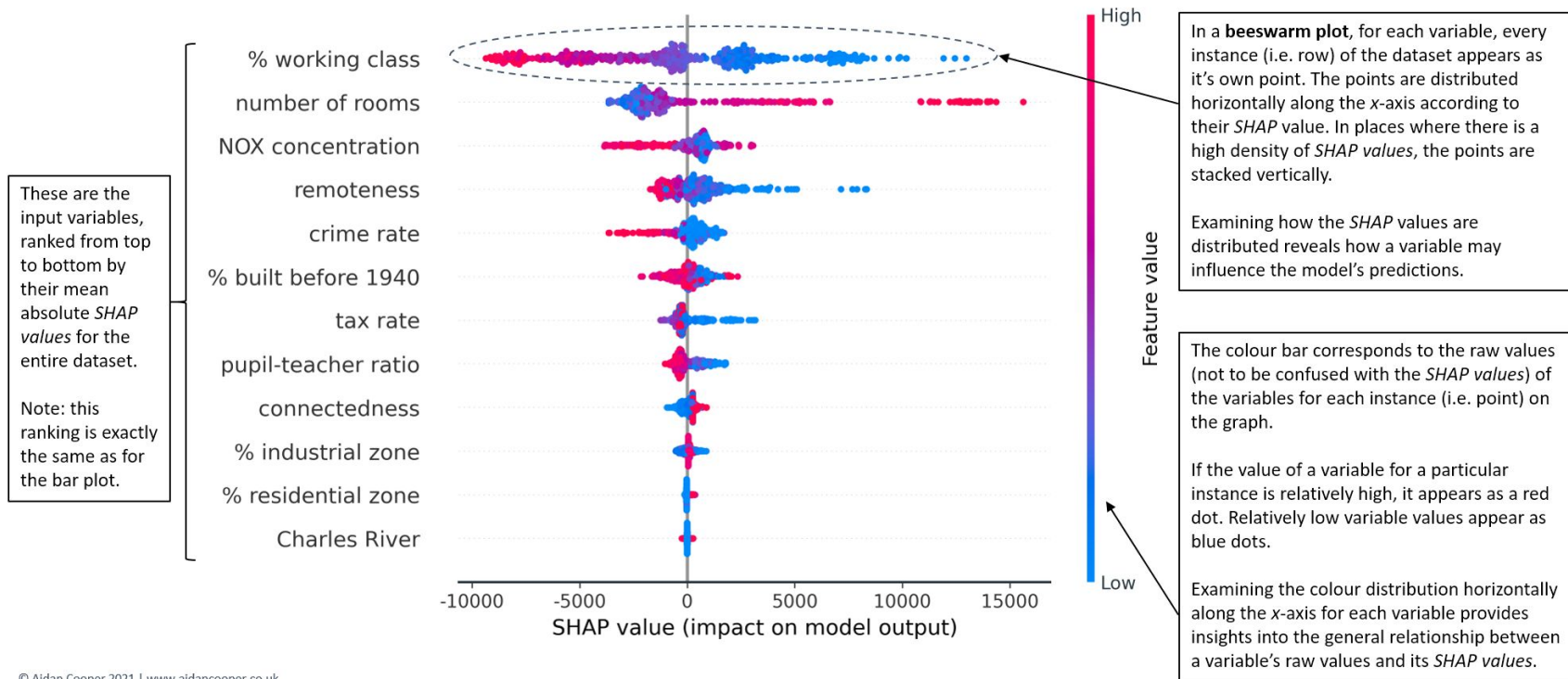
Global Explanation (Beeswarm Plot): Used to explain the *entire model*.

- Every dot is a single prediction for a single patient.
- The X-axis is the SHAP value (impact on model output).
- The Color represents the actual feature value (e.g., Red = High Age, Blue = Low Age).
- If a feature has a long tail of red dots stretching to the right, it definitively proves that high values of that feature drastically increase the model's prediction.

Force plot



Beeswarm plot



Gradient Boosting

Let's play a game...

You are given $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, and the task is to fit a model $F(x)$ to minimize square loss. Suppose your friend wants to help you and gives you a model F . You check his model and find the model is good but not perfect. There are some mistakes: $F(x_1)=0.8$, while $y_1 = 0.9$, and $F(x_2)=1.4$ while $y_2 = 1.3$... How can you improve this model?

Rule of the game:

- You are not allowed to remove anything from F or change any parameter in F .
- You can add an additional model (regression tree) h to F , so the new prediction will be $F(x) + h(x)$.

Thus we get the following:

$$\begin{array}{ll} F(x_1) + h(x_1) = y_1 & h(x_1) = y_1 - F(x_1) \\ F(x_2) + h(x_2) = y_2 & h(x_2) = y_2 - F(x_2) \\ \dots & \dots \\ F(x_n) + h(x_n) = y_n & h(x_n) = y_n - F(x_n) \end{array} \quad \Rightarrow$$

Gradient Boosting

Relation to Gradient Descent

To achieve this we can just fit a tree to the following data: $(x_1, y_1 - F(x_1)), (x_2, y_2 - F(x_2)), \dots, (x_n, y_n - F(x_n))$. $y_n - F(x_n)$ are called residuals. The role of h is to compensate the shortcoming of existing model F . If the new model $F + h$ is still not satisfactory, we can add another regression tree...

Our loss function is given as: $L(y, F(x)) = \frac{1}{2} * (y - F(x))^2$

We want to minimize $J = \sum_i L(y_i, F(x_i))$ by adjusting $F(x_1), F(x_2), \dots, F(x_n)$. Notice that $F(x_1), F(x_2), \dots, F(x_n)$ are just some numbers. We can treat $F(x_i)$ as parameters and take derivatives:

$$\frac{\partial J}{\partial F(x_i)} = \frac{\partial \sum_i L(y_i, F(x_i))}{\partial F(x_i)} = \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = F(x_i) - y_i$$

So we can interpret residuals as negative gradients.

$$y_i - F(x_i) = -\frac{\partial J}{\partial F(x_i)} \quad F(x_i) := F(x_i) - 1 \frac{\partial J}{\partial F(x_i)}$$
$$\theta_i := \theta_i - \rho \frac{\partial J}{\partial \theta_i}$$

XGBoost Regression

Main Algorithm:

Step 1: We are given input features (X) and target feature (Y). Now we start with a default set of predictions (by default set to 0.5 in both classification and regression but you can start from other values as well). Then we calculate our **Pseudo Residuals**.

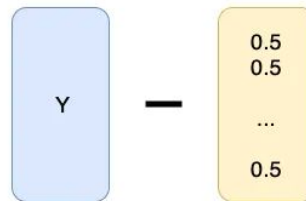
In general:

$$r_i = -\frac{\partial \mathcal{L}(y_i, \hat{y}_i)}{\partial \hat{y}_i}$$

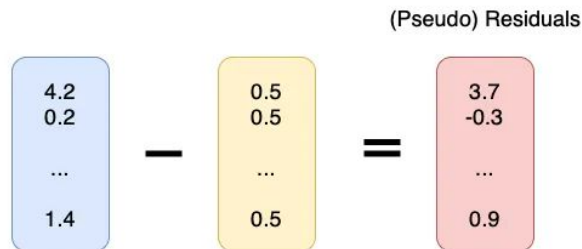
For MSE:

$$r_i = y_i - \hat{y}_i$$

Target Feature Default initial preds



That is ...



XGBoost Regression

Main Algorithm:

Step 2: We then need to build the XGBoost trees:

Let's start with all residuals in the same leaf: **-10.5, 6.5, 7.5, -7.5**

We calculate our similarity score using the gradients g_i and Hessians h_i :

In general:
$$\text{Score} = \frac{(\sum g_i)^2}{\sum h_i + \lambda}$$

For MSE: Similarity Score =
$$\frac{(\sum residuals)^2}{len(residuals) + \lambda}$$


Regularization parameter

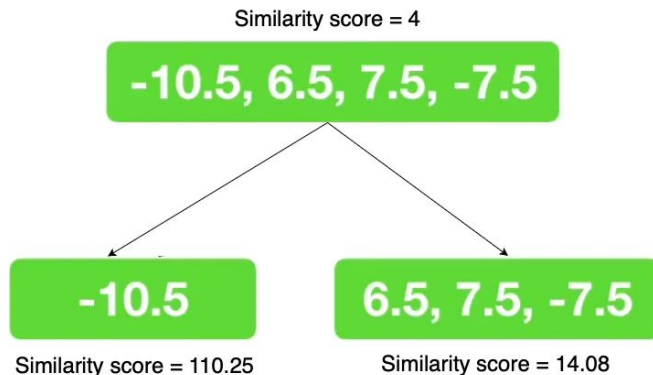
Step 3: Now we go through all the input features one by one.

- ❑ Select the first feature, sort its values in ascending order and go through those values one by one.
- ❑ Take 2 points at a time from the start, get the mean of those 2 values and then divide the leaf residuals according to the mean value, i.e. put residuals of elements with less than mean feature value to one node and the others to a different node.
- ❑ Do this for all features and all values in each feature.

XGBoost Regression

Main Algorithm:

Step 4: We now calculate the gain value for each of the splits created in step 3 as shown below. We go through all of the splits in step 3 and then take the split which gave us the highest gain. i.e. we select the one which best splits the observations.



Now calculate Gain for this split

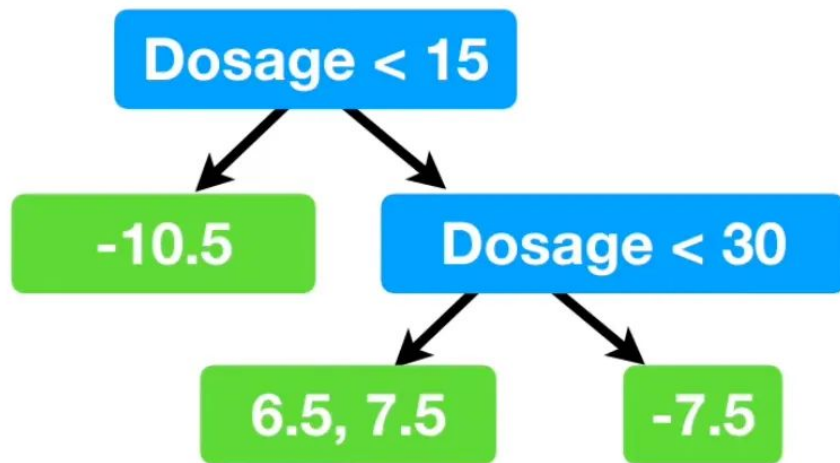
$$\text{Gain} = \text{ss}(\text{left_child}) + \text{ss}(\text{right_child}) - \text{ss}(\text{Root})$$

$$\text{Gain} = 110.25 + 14.08 - 4 = 120.33$$

XGBoost Regression

Main Algorithm:

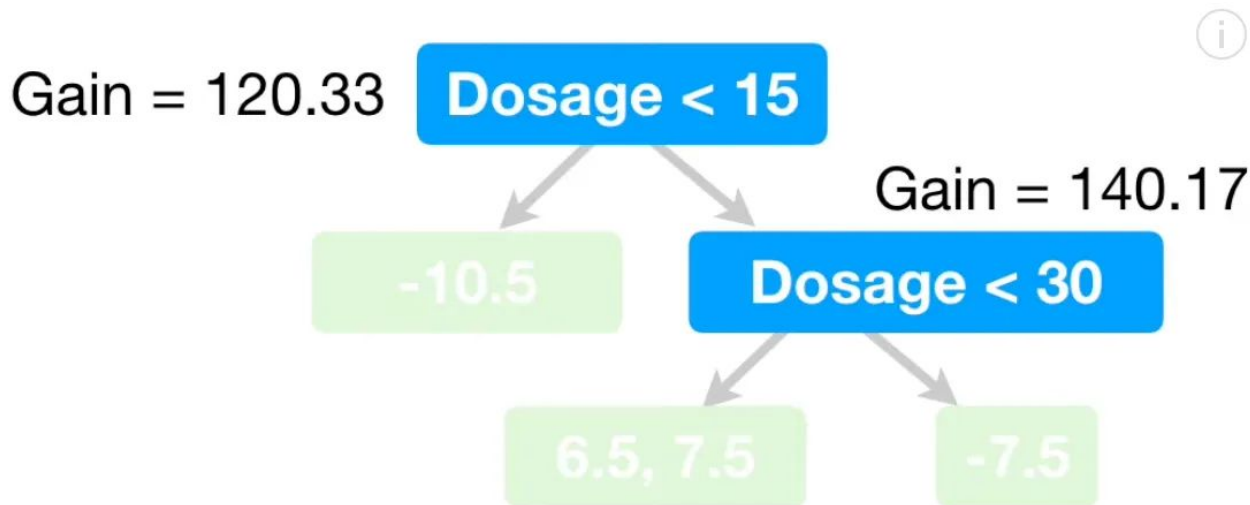
Step 5: We continue doing step 3 & 4 for the children of the tree to split it further. The stopping conditions are either the max depth has been reached (6 by default) or the leaf has just minimum amount of residuals in it (default = 1). So after some time we may get a tree like this.



XGBoost Regression

Main Algorithm:

Step 6: We use a hyper parameter γ to prune trees. We go bottom up from the tree while pruning. If the Gain of a parent node is less than the γ value then we prune its children (more formally if $\text{gain} - \gamma < 0$). We only go up if we pruned at that particular point else we stop there.



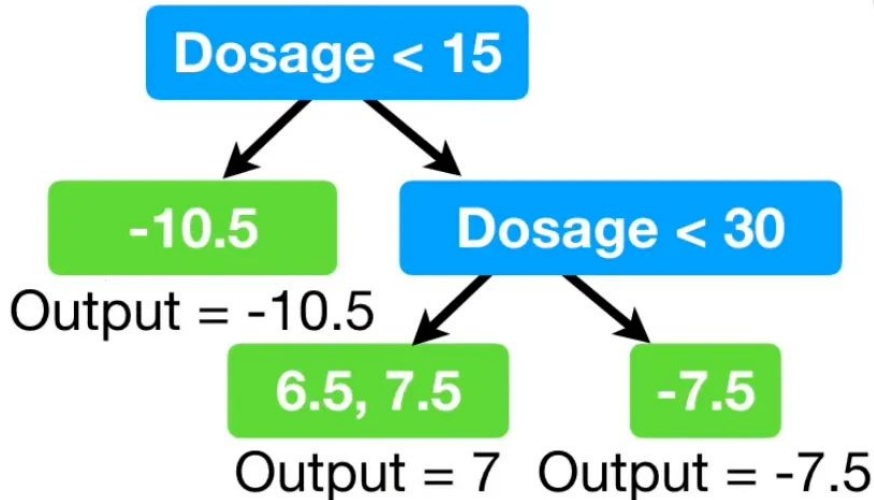
XGBoost Regression

Main Algorithm:

Step 7: Now we'll calculate the output values for each of the child nodes in the tree using the following formula.

$$\text{Output Value} = \frac{\sum residuals}{len(residuals) + \lambda}$$

$\downarrow$
Regularization parameter
(default 0)



XGBoost Regression

Main Algorithm:

Step 8: Now that we have a tree that can predict residuals, we can update our initial default vector of 0.5 predictions by adding these residuals to it (multiplied by a learning rate of course. We don't want to directly go to the pred value and overfit).

Predicted Drug
Effectiveness

0.5

+ Learning Rate **X**

Dosage < 15

-10.5

Output = -10.5

Dosage < 30

6.5, 7.5

Output = 7

-7.5

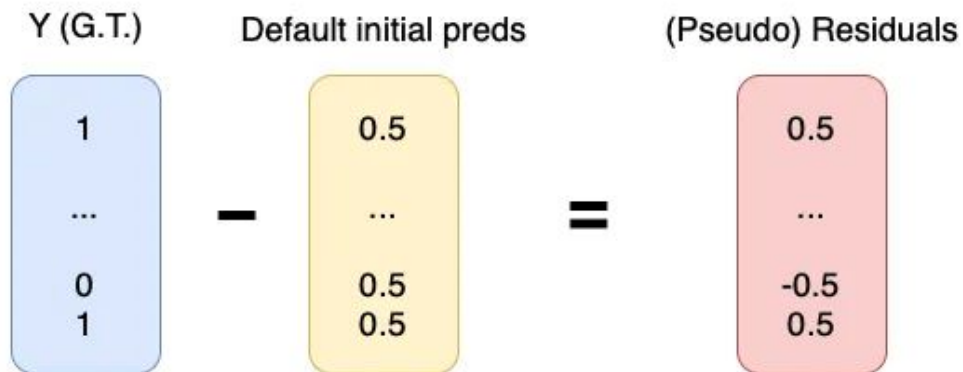
Output = -7.5

XGBoost calls this learning
rate 'eta' (default 0.3)

XGBoost Classification

Main Algorithm:

Step 1: Given the data and initial predictions we again calculate the pseudo-residuals.



Step 2: Starting with all residuals in the same leaf, we will calculate the similarity score considering:

$$\text{Similarity Score} = \frac{(\sum residuals)^2}{\sum P_{i-1} * (1 - P_{i-1}) + \lambda}$$

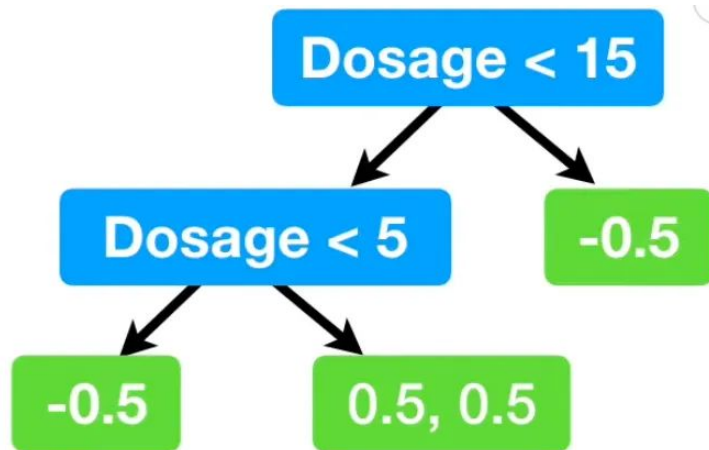
XGBoost Classification

Main Algorithm:

Step 3: We find the best splitting feature and its values. (Same as regression) Go through all of the features one by one. Then sort all values in a feature and go through the values one by one. Find the mean of 2 values at a time and split leaf values according to that value.

Step 4: We calculate the Gain (Same as regression), finding the split with the highest gain.

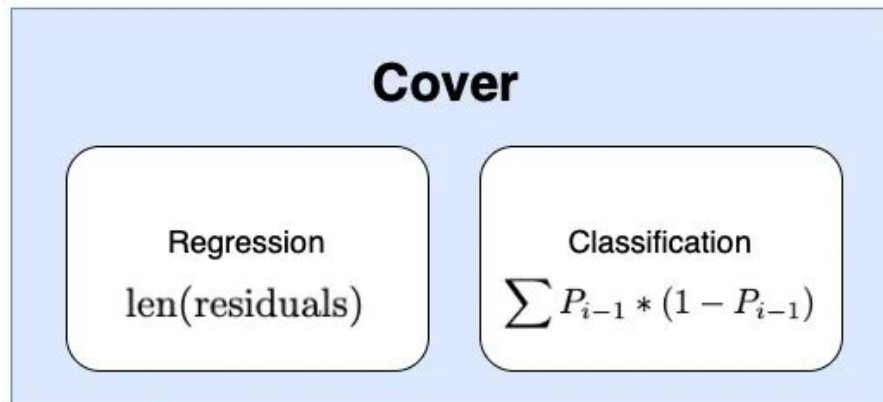
Step 5: We keep splitting the nodes in your nodes according to steps 3,4 to build the tree.



XGBoost Classification

What is cover?

Cover is basically defined as the denominator of similarity score minus lambda. So basically the formula is:



By default the value of cover is set to 1 in both cases.

- **Regression:** Since the minimum value of cover is 1 by default and we can't have less than 1 residuals in a leaf cover has no effect on how we grow the tree for the default value of cover.
- **Classification:** If the calculated cover value for a leaf is less than 1 then we discard that division (essentially pruning the left).

XGBoost Classification

Main Algorithm:

Step 6: We prune our tree. Start bottom up and just prune based on the hyper-parameter gamma (γ) and gain for each node.

Step 7: We calculate the output value at each node. As you can see the formula quite similar to what we saw in Regression.

$$\text{Output Value} = \frac{\sum(\text{residuals})}{\sum P_{i-1} * (1 - P_{i-1}) + \lambda}$$

Since we can not directly add these values (multiplied by learning rate(η)) as this is a classification task, we need to convert the initial probability (0.5) into $\log(\text{odds})$, perform the addition with η multiplied and convert it back to probability.

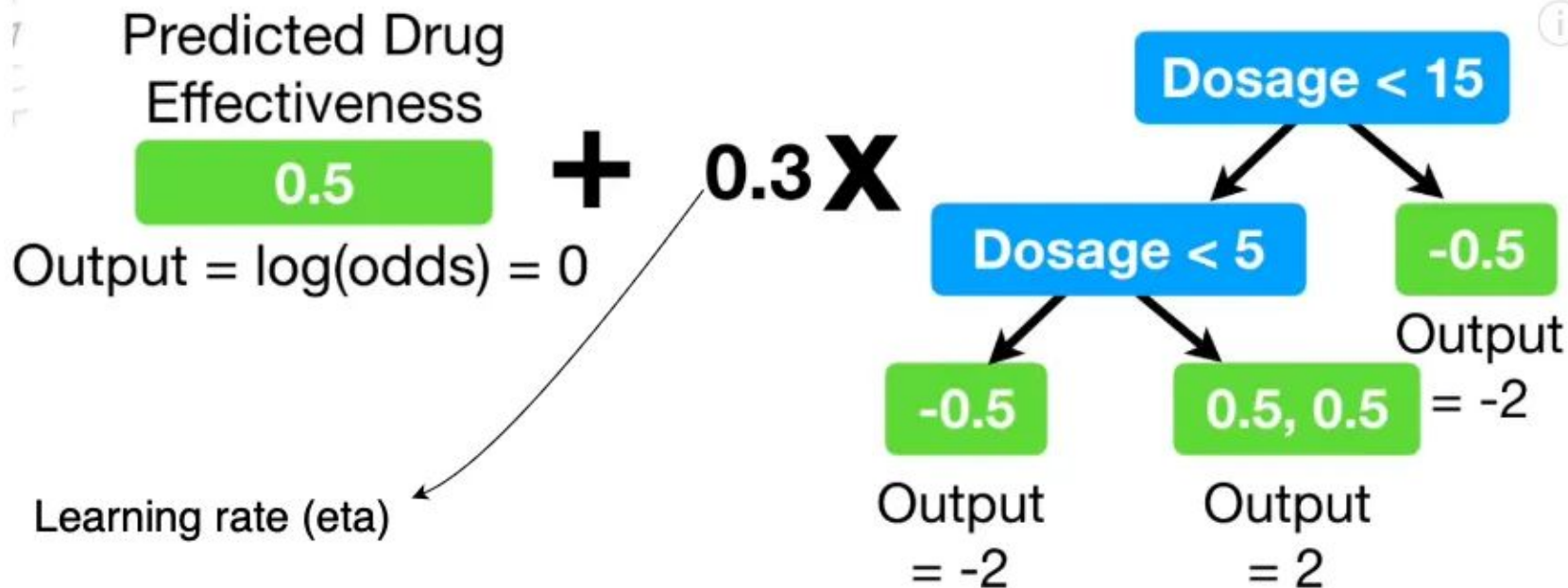
$$\frac{p}{1-p} = \text{odds}$$

$$\log\left(\frac{p}{1-p}\right) = \log(\text{odds})$$

$$\text{Probability} = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$

XGBoost Classification

Main Algorithm:



XGBoost: Mathematical Details

How do we get the following formulae for our similarity score and output value?

Regression

$$\text{Similarity Score} = \frac{\text{Sum of Residuals, Squared}}{\text{Number of Residuals} + \lambda}$$

VS

Classification

$$\frac{(\sum \text{Residual}_i)^2}{\sum [P_i \times (1 - P_i) + \lambda]}$$

$$\text{Output Value} = \frac{\text{Sum of Residuals}}{\text{Number of Residuals} + \lambda}$$

VS

$$\frac{(\sum \text{Residual}_i)}{\sum [P_i \times (1 - P_i) + \lambda]}$$

XGBoost: Mathematical Details

$$\text{Loss} = \left[\sum_{i=1}^n L(y_i, p_i^0 + O_{\text{value}}) \right] + \frac{1}{2} \lambda O_{\text{value}}^2$$

Note, by Tolor series expansion we have the following expansion

$$L(y, p_i + O_{\text{value}}) \approx L(y, p_i) + \left[\frac{d}{dp_i} L(y, p_i) \right] O_{\text{value}} + \frac{1}{2} \left[\frac{d^2}{dp_i^2} L(y, p_i) \right] O_{\text{value}}^2$$

First derivative is
called gradient (g)

Second derivative is
called hessian (h)

$$L(y, p_i + O_{\text{value}}) \approx L(y, p_i) + g O_{\text{value}} + \frac{1}{2} h O_{\text{value}}^2$$

XGBoost: Mathematical Details

So the plugging in the second order taylor approximation in the loss function we would get...

$$\begin{aligned}\text{Loss} &= L(y_1, p_1^0) + g_1 O_{\text{value}} + \frac{1}{2} h_1 O_{\text{value}}^2 \\ &\quad + L(y_2, p_2^0) + g_2 O_{\text{value}} + \frac{1}{2} h_2 O_{\text{value}}^2 + \dots + \\ &\quad + L(y_n, p_n^0) + g_n O_{\text{value}} + \frac{1}{2} h_n O_{\text{value}}^2 + \frac{1}{2} \lambda O_{\text{value}}^2 \\ &= \sum_{i=1}^n L(y_i, p_i^0) + O_{\text{val}} \sum_{i=1}^n g_i + \frac{1}{2} O_{\text{val}}^2 \sum_{i=1}^n h_i + \frac{1}{2} \lambda O_{\text{val}}^2\end{aligned}$$

XGBoost: Mathematical Details

Now let's do what we do when we want to minimize a function! Take the derivative and set it to 0

$$0 + \sum_{i=1}^n g_i + O_{val} \sum_{i=1}^n h_i + \lambda O_{val} = 0$$

Solving this further we get, ...

$$O_{val} = \frac{-\sum_{i=1}^n g_i}{[\sum_{i=1}^n h_i] + \lambda}$$

XGBoost: Mathematical Details

For Regression in XGB the most common loss function is

$$L(y_i, p_i) = \frac{1}{2}(y_i - p_i)^2$$

g_i and h_i are basically the first and second derivatives of this loss function.
So solving for those ...

$$g_i = \frac{d}{dp_i} \frac{1}{2}(y_i - p_i)^2 = -(y_i - p_i)$$

$$h_i = \frac{d^2}{dp_i^2} \frac{1}{2}(y_i - p_i)^2 = 1$$

XGBoost: Mathematical Details

Substituting g_i and h_i

$$O_{val} = \frac{-\sum_{i=1}^n -(y_i - p_i)}{\sum_{i=1}^n 1 + \lambda} = \frac{\sum_{i=1}^n (y_i - p_i)}{n + \lambda}$$

Therefore,

Output Value	=	$\frac{\text{Sum of Residuals}}{\text{Number of Residuals} + \lambda}$
--------------	---	--

XGBoost: Mathematical Details

We can start directly from this step since only the loss function is different in classification.

$$O_{val} = \frac{-\sum_{i=1}^n g_i}{[\sum_{i=1}^n h_i] + \lambda}$$

For Classification in XGB the most common loss function is

$$L(y_i, p_i) = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

$$L(y_i, \log(\text{odds})_i) = -y_i \log(\text{odds}) + \log(1 + e^{\log(\text{odds})})$$

XGBoost: Mathematical Details

$$g_i = \frac{d}{d \log(\text{odds})} L(y_i, \log(\text{odds})_i) = -y_i + \frac{e^{\log(\text{odds})_i}}{1 + e^{\log(\text{odds})_i}} = -(y_i - p_i)$$

$$h_i = \frac{d^2}{d \log(\text{odds})^2} L(y_i, \log(\text{odds})_i) = \frac{e^{\log(\text{odds})}}{(1 + e^{\log(\text{odds})})} \times \frac{1}{(1 + e^{\log(\text{odds})})} = p_i \times (1 - p_i)$$

Substituting g_i and h_i

$$O_{val} = \frac{-\sum_{i=1}^n -(y_i - p_i)}{[\sum_{i=1}^n p_i * (1 - p_i)] + \lambda} = \frac{\sum_{i=1}^n (y_i - p_i)}{[\sum_{i=1}^n p_i * (1 - p_i)] + \lambda}$$

XGBoost: Mathematical Details

Substituting g_j and h_j

$$O_{val} = \frac{-\sum_{i=1}^n -(y_i - p_i)}{[\sum_{i=1}^n p_i * (1 - p_i)] + \lambda} = \frac{\sum_{i=1}^n (y_i - p_i)}{[\sum_{i=1}^n p_i * (1 - p_i)] + \lambda}$$

Therefore,

$$\text{Output Value} = \frac{(\sum \text{Residual}_i)}{\sum [P_i \times (1 - P_i) + \lambda]}$$

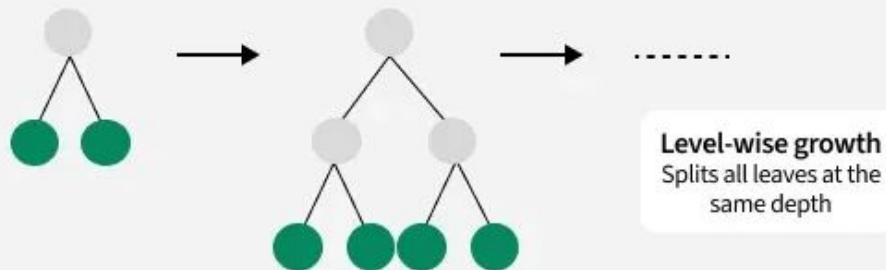
LightGBM

Speeding up the Gradient

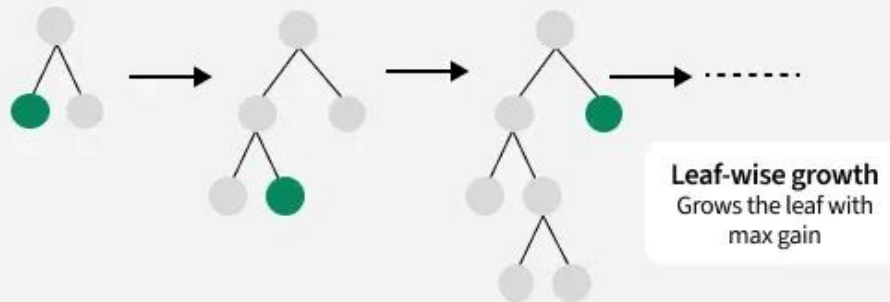
- **The XGBoost Bottleneck:** Traditional XGBoost grows trees **Level-Wise** (Depth-first). It completely fills out Layer 1, then Layer 2, calculating splits for every single node across the entire level.
- **The LightGBM Solution:** Developed by Microsoft, LightGBM grows trees **Leaf-Wise** (Best-first). It looks at all current leaves, finds the single leaf that will reduce the global loss the most, and splits *only* that leaf.
- **The Result:** LightGBM creates highly asymmetrical trees. It is drastically faster than XGBoost and uses far less memory, making it the industry standard for massive datasets.
- **The Catch:** Because Leaf-Wise growth is so aggressive, LightGBM is highly prone to overfitting on small datasets (<10,000 rows).

Illustration

XGBoost



LightGBM



Model Comparison

Which model to use?

Random Forest:

- *Pros:* Highly parallelizable (fast to train), incredibly robust to overfitting, requires almost zero hyperparameter tuning. It works "out of the box."
- *Cons:* Can result in massive file sizes, struggles with highly imbalanced data.
- *When to use:* As your baseline model for every tabular dataset.

XGBoost:

- *Pros:* Consistently wins Kaggle competitions. Natively handles missing values (NaNs). Best raw predictive power.
- *Cons:* Sequential training is slower. Highly sensitive to hyperparameters (Learning Rate, Max Depth); it *will* overfit if you don't tune it properly.
- *When to use:* When you need maximum accuracy and have time to run GridSearch tuning.

LightGBM:

- *Pros:* Blisteringly fast. Very low RAM usage.
- *Cons:* Overfits aggressively on small datasets.
- *When to use:* When your dataset has hundreds of thousands or millions of rows.

Other Ensemble Models

- **Extra Trees (Extremely Randomized Trees)** – Like Random Forests, but split thresholds are chosen randomly, increasing bias slightly while reducing variance and speeding up training.
- **CatBoost** – Gradient boosting that natively handles categorical features using ordered target statistics and reduces target leakage.
- **Stacking (Stacked Generalization)** – Combines predictions from multiple different models using a meta-model trained on their outputs.
- **Blending** – A simplified form of stacking where the meta-model is trained on a hold-out validation set instead of cross-validation.
- **Voting Classifier / Regressor** – Aggregates predictions from multiple models via majority vote (classification) or averaging (regression).
- **Bagged k-NN** – Applies bootstrap aggregation to k-nearest neighbors to reduce variance and improve stability.
- **Random Subspace Method** – Trains models on random subsets of features rather than samples to increase diversity.
- **Gradient Boosted Decision Trees (GBDT)** – Generic boosting framework that sequentially fits trees to residuals of previous models.
- **AdaBoost.M1 / AdaBoost.R2** – Variants of AdaBoost specialized for classification and regression respectively.
- **Rotation Forest** – Trains decision trees on rotated feature spaces obtained via PCA to increase diversity while preserving information.

Useful links

- <https://medium.com/@eng.mohammed.saad.18/detailed-explanation-of-random-forests-features-importance-bias-8755d26ac3bc>
- <https://www.youtube.com/watch?v=3CC4N4z3GJc>
- <https://www.youtube.com/watch?v=2xudPOBz-vs>
- <https://www.youtube.com/watch?v=jxuNLH5dXC&t=798s>
- <https://www.youtube.com/watch?v=StWY5QWMXCw>
- <https://www.youtube.com/watch?v=OtD8wVaFm6E>
- <https://www.youtube.com/watch?v=8b1JEDvenQU>
- <https://www.youtube.com/watch?v=ZVFeW798-2I>
- <https://www.youtube.com/watch?v=oRrKeUCEbq8>
- <https://www.youtube.com/watch?v=LsK-xG1cLYA>